

Enterprise AI Knowledge Corpus for RAG Systems

A substantive reference document across five domains — Artificial Intelligence Fundamentals, Vector Databases, Retrieval-Augmented Generation, Cloud Infrastructure, and Cybersecurity — structured in ten knowledge cycles for RAG benchmarking and experimentation.

Enriched Edition — 5 Knowledge Cycles × 5 Domains

Knowledge Cycle 1

Artificial Intelligence Fundamentals

AI Fundamentals – Origins and Core Concepts

Artificial Intelligence (AI) refers to the simulation of human cognitive processes—reasoning, learning, perception, and problem-solving—by computer systems. The field was formally founded at the Dartmouth Conference in 1956, where researchers including John McCarthy, Marvin Minsky, and Claude Shannon gathered to explore whether machines could be made to reason like humans.

Early AI systems relied on explicit rules encoded by human experts, a paradigm known as symbolic AI or Good Old-Fashioned AI (GOFAI). Expert systems such as MYCIN (1970s) and XCON (1980s) demonstrated that narrow, rule-based reasoning could perform at expert human levels in constrained domains. However, the knowledge-acquisition bottleneck and the brittleness of hand-crafted rules limited their broader applicability.

The shift toward machine learning (ML) marked a fundamental change in methodology: instead of encoding rules, practitioners allowed systems to infer rules from data. Statistical learning algorithms such as decision trees, support vector machines, and Naive Bayes classifiers became the dominant approach through the 1990s and 2000s. The landmark win of IBM's Deep Blue against chess champion Garry Kasparov in 1997 captured public attention, even though Deep Blue relied primarily on brute-force search rather than learned heuristics.

The deep learning revolution, catalysed by the use of GPUs for training large neural networks, began in earnest around 2012 when AlexNet won the ImageNet competition by a wide margin. Convolutional Neural Networks (CNNs) became the standard for vision tasks, and Recurrent Neural Networks (RNNs) with Long Short-Term Memory (LSTM) cells advanced natural language processing. Today, AI is embedded across industries: fraud detection in banking, diagnostic imaging in medicine, demand forecasting in logistics, and code generation in software engineering.

Vector Databases

Vector Databases – What They Are and Why They Exist

A vector database is a specialised data store designed to index and query high-dimensional numerical vectors efficiently. These vectors—commonly called embeddings—are dense numerical representations of real-world objects: words, sentences, images, audio clips, or even structured records. Two objects that are semantically similar will have embeddings that are numerically close in the vector space, typically measured by cosine similarity or Euclidean distance.

Traditional relational databases excel at exact-match and range queries over structured data. However, a query like 'find me documents semantically similar to this paragraph' cannot be expressed efficiently in SQL without specialised extensions. Vector databases fill this gap by implementing Approximate Nearest Neighbour (ANN) algorithms—such as HNSW (Hierarchical Navigable Small World graphs), IVF (Inverted File Index), and PQ (Product Quantisation)—that find the closest vectors in sub-linear time.

The need for vector databases exploded alongside the adoption of transformer-based embedding models such as OpenAI's text-embedding-ada-002, Sentence-BERT, and Cohere Embed. Every RAG system, semantic search engine, recommendation system, and anomaly detector that relies on embedding similarity needs a scalable, low-latency way to store and retrieve those embeddings at production scale.

Key capabilities expected of modern vector databases include: real-time upserts without full re-indexing, metadata filtering to combine semantic and structured queries, multi-tenancy for data isolation, horizontal scaling across nodes, and optional persistence on durable storage. Popular choices include Pinecone (fully managed SaaS), Weaviate (open-source, GraphQL API), Qdrant (Rust-native, high performance), FAISS (in-memory library from Meta), and ChromaDB (lightweight, developer-friendly).

Retrieval-Augmented Generation

RAG – The Architecture and Its Motivation

Large language models (LLMs) are trained on static corpora with fixed knowledge cutoff dates. When deployed, they cannot access information created after training, they cannot retrieve proprietary enterprise data, and they are prone to hallucination when asked about facts they were never trained on. Retrieval-Augmented Generation (RAG) addresses these limitations by connecting a generative model to a dynamic knowledge store at inference time.

The RAG architecture, first formalised by Lewis et al. at Meta AI Research in 2020, has two components working in sequence. The retriever searches an external corpus for documents relevant to a given query, typically using dense vector similarity. The generator—an LLM—is then prompted with both the original query and the retrieved documents as context, producing a response grounded in retrieved evidence rather than parametric memory alone.

This approach has several practical advantages. It allows knowledge to be updated without retraining the LLM—simply refreshing the vector store is sufficient. It provides an audit trail: the source documents used to generate a response can be cited explicitly, improving transparency. It also reduces hallucination because the model is conditioned on retrieved facts rather than relying solely on what it learned during pre-training.

RAG has quickly become the dominant pattern for enterprise AI applications: internal knowledge bases, legal research assistants, customer support bots, and medical reference tools. The quality of a RAG system depends critically on three factors: the quality and freshness of the indexed corpus, the accuracy of the retrieval step, and the ability of the LLM to synthesise retrieved context faithfully.

Cloud Infrastructure

Cloud Infrastructure – Foundations of Modern Distributed Computing

Cloud computing delivers computing resources—servers, storage, databases, networking, software—over the internet on a pay-per-use basis. The three primary service models are Infrastructure-as-a-Service (IaaS), Platform-as-a-Service (PaaS), and Software-as-a-Service (SaaS). Amazon Web Services (AWS), Microsoft Azure, and Google Cloud Platform (GCP) collectively capture over 65% of global cloud market share.

The physical backbone of cloud infrastructure consists of massive data centres distributed across geographic regions. Each cloud provider divides its global footprint into Regions (large geographic areas) and Availability Zones (AZs)—isolated data-centre clusters within a region. Deploying workloads across multiple AZs provides fault tolerance; deploying across multiple regions provides disaster recovery and reduced latency for globally distributed users.

Virtualisation is the enabling technology that allows physical servers to be partitioned into many independent virtual machines (VMs). Hypervisors such as KVM, Xen, and Hyper-V abstract the hardware and allocate slices of compute, memory, and storage to each VM. Cloud providers have further refined this with bare-metal instances (no hypervisor overhead), spot/preemptible instances (unused capacity at steep discounts), and serverless functions (event-driven compute with no server management).

Networking in the cloud is defined by Virtual Private Clouds (VPCs), subnets, security groups (stateful firewalls), network ACLs, load balancers, and content delivery networks (CDNs). Infrastructure-as-Code (IaC) tools—Terraform, AWS CloudFormation, Pulumi—allow teams to declare and version their entire cloud topology in code, enabling repeatable, auditable deployments. CI/CD pipelines automate the path from code commit to production deployment, typically gated by automated tests and approval workflows.

Cybersecurity and Governance

Cybersecurity – Threat Landscape and Foundational Principles

Cybersecurity is the discipline of protecting computer systems, networks, and data from unauthorised access, damage, or disruption. The threat landscape is vast and constantly evolving: adversaries

include nation-state actors, organised crime syndicates, hacktivists, and insiders. Attack categories include malware (viruses, ransomware, spyware), phishing and social engineering, denial-of-service (DoS/DDoS), man-in-the-middle attacks, SQL injection, and supply-chain compromises.

The CIA Triad—Confidentiality, Integrity, and Availability—is the foundational model for evaluating security controls. Confidentiality ensures that data is accessible only to authorised parties (achieved via encryption, access controls). Integrity ensures that data is accurate and has not been tampered with (achieved via hashing, digital signatures). Availability ensures that systems and data are accessible to authorised users when needed (achieved via redundancy, backups, DDoS mitigation).

Defence-in-depth is the principle of layering multiple independent security controls so that if one layer is compromised, others still protect the system. A typical enterprise security stack includes perimeter firewalls, intrusion detection/prevention systems (IDS/IPS), endpoint detection and response (EDR) agents, privileged access management (PAM), data loss prevention (DLP) tools, and a Security Information and Event Management (SIEM) platform that aggregates and correlates logs from across the environment.

Zero Trust Architecture (ZTA) represents a modern shift away from the traditional 'castle-and-moat' model. Under ZTA, no user, device, or network segment is trusted by default—even those inside the corporate perimeter. Every access request must be authenticated, authorised, and continuously validated. Key enablers include identity providers (IdPs) like Okta and Azure AD, multi-factor authentication (MFA), micro-segmentation, and just-in-time access provisioning.

Knowledge Cycle 2

Artificial Intelligence Fundamentals

AI Fundamentals – Neural Networks and Deep Learning

Artificial neural networks (ANNs) are computational models loosely inspired by the biological neural networks in animal brains. A network consists of layers of nodes (neurons), each of which applies a weighted linear transformation to its inputs followed by a non-linear activation function such as ReLU, sigmoid, or tanh. The composition of many such layers enables the network to learn hierarchical representations of complex data.

Training a neural network requires a labelled dataset, a loss function that measures prediction error, and an optimisation algorithm. Stochastic Gradient Descent (SGD) and its variants—Adam, RMSProp, AdaGrad—compute gradients of the loss with respect to network weights via backpropagation and iteratively update weights to minimise loss. Regularisation techniques (dropout, weight decay, batch normalisation) prevent overfitting.

Deep learning refers to neural networks with many layers (typically more than three). Convolutional Neural Networks (CNNs) exploit spatial locality and translational invariance, making them ideal for image tasks. Recurrent Neural Networks (RNNs) and LSTMs process sequential data by maintaining hidden state across time steps. Graph Neural Networks (GNNs) operate on graph-structured data, enabling applications in drug discovery, social network analysis, and knowledge graph reasoning.

Hardware acceleration has been decisive in enabling deep learning at scale. Graphics Processing Units (GPUs) offer thousands of parallel cores optimised for the matrix multiplications that dominate neural network computation. Tensor Processing Units (TPUs), Google's custom ASICs, provide further efficiency for large-scale training. Distributed training strategies—data parallelism, model parallelism, pipeline parallelism—spread computation across hundreds or thousands of accelerators, enabling the training of models with hundreds of billions of parameters.

Vector Databases

Vector Databases – Indexing Algorithms

The efficiency of a vector database depends almost entirely on its indexing algorithm. Brute-force exact nearest-neighbour search requires computing the distance from the query vector to every stored vector, which is $O(n \cdot d)$ where n is the number of vectors and d is the dimensionality. For collections with millions or billions of vectors at hundreds of dimensions, this is computationally prohibitive at real-time latencies.

HNSW (Hierarchical Navigable Small World) is currently the most popular ANN index. It builds a multi-layer graph where each node represents a vector and edges connect nearby vectors. The top layers are sparse (long-range connections for fast navigation); lower layers are dense (fine-grained similarity search). Queries traverse the graph top-down, greedily following the neighbour closest to the query at each layer. HNSW achieves excellent recall-speed trade-offs and supports incremental insertions without full re-indexing.

IVF (Inverted File Index) clusters vectors into Voronoi cells using k-means. At query time, only a subset of cells (nprobe) is searched, reducing computation. IVF is often combined with Product Quantisation (PQ), which compresses vectors by splitting them into sub-vectors and encoding each sub-vector with a codebook, dramatically reducing memory footprint. The combined IVF-PQ scheme is used in FAISS and enables billion-scale search on commodity hardware.

ScaNN (Scalable Nearest Neighbours), developed by Google Research, uses anisotropic quantisation that prioritises preserving the components of vectors most relevant to maximum inner product search (MIPS). DiskANN stores the HNSW graph on SSD rather than RAM, enabling single-machine search over billion-scale datasets that would otherwise require expensive high-memory servers. Choosing the right index involves balancing recall, latency, throughput, memory usage, and build time.

Retrieval-Augmented Generation

RAG – Chunking Strategies in Depth

Chunking—dividing source documents into smaller units before indexing—is one of the highest-leverage decisions in RAG system design. The chunk is the atomic unit of retrieval; whatever is stored in the vector index will be returned in full when matched. If chunks are too large, retrieved context is noisy and may exceed the LLM's context window. If chunks are too small, they lack sufficient context for the model to formulate a coherent answer.

Fixed-size character chunking splits text every N characters, with an optional overlap of M characters between consecutive chunks. It is simple and fast but ignores linguistic structure: a chunk may begin mid-sentence or split a table. Recursive character splitting improves on this by trying a sequence of separators (double newline, single newline, period, space) and recursively splitting on the next separator when a chunk is still too large. LangChain's RecursiveCharacterTextSplitter implements this approach.

Semantic chunking uses an embedding model to detect topic boundaries dynamically. Sentences are embedded, and a chunk boundary is placed wherever the cosine similarity between adjacent sentence embeddings drops below a threshold—signalling a change in topic. This produces chunks of variable length that better respect semantic coherence. The trade-off is higher CPU cost at index build time and sensitivity to the threshold hyperparameter.

Sentence-window retrieval is a hybrid strategy: the index stores individual sentences, but at retrieval time the system returns the surrounding window of sentences (e.g., the two sentences before and after the matched sentence) as context to the LLM. This maximises retrieval precision while providing the LLM with sufficient local context. Parent-document retrieval goes further: small chunks are indexed, but the retriever returns the full parent document or section when a chunk matches, giving the model broad context.

Cloud Infrastructure

Cloud Infrastructure – Containerisation and Orchestration

Containers package an application and all its dependencies—libraries, runtime, configuration—into a single, portable image that runs identically across development laptops, CI pipelines, and production servers. Docker popularised the container format; an image is built from a Dockerfile specifying the base OS layer, software packages, and application code. Containers share the host OS kernel, making them far lighter than virtual machines while maintaining process-level isolation.

As organisations began running hundreds or thousands of containers, the need for automated scheduling, scaling, health checking, and networking became apparent. Kubernetes (K8s), open-sourced by Google in 2014, became the de facto container orchestration standard. Kubernetes models applications as Pods (the smallest deployable unit, containing one or more containers), organised into Deployments (stateless workloads), StatefulSets (databases, queues), and DaemonSets (per-node agents).

Kubernetes introduces several powerful abstractions: Services provide stable DNS names and load-balanced IP addresses for sets of Pods; Ingress controllers manage external HTTP/HTTPS routing; ConfigMaps and Secrets externalise configuration and credentials; PersistentVolumeClaims provision durable storage from cloud block or file storage. The Horizontal Pod Autoscaler (HPA) scales Deployments based on CPU, memory, or custom metrics, while the Cluster Autoscaler adds or removes nodes based on pending workloads.

Service meshes—Istio, Linkerd, Cilium—add a layer of infrastructure for observability, traffic management, and mutual TLS between services without changing application code. Each Pod gets a sidecar proxy (Envoy) that intercepts all network traffic, enabling fine-grained policies: retries, circuit breakers, canary traffic splits, and distributed tracing. Managed Kubernetes offerings (EKS, GKE, AKS) handle control-plane operations, leaving teams to focus on workload configuration.

Cybersecurity and Governance

Cybersecurity – Identity and Access Management

Identity and Access Management (IAM) is the discipline of ensuring that the right entities (humans, service accounts, devices) have the right level of access to the right resources at the right time—and no more. IAM is often cited as the single most impactful security control: the majority of data breaches involve compromised credentials or excessive privileges.

Authentication verifies identity. Modern enterprise authentication relies on federated identity: employees authenticate once with a central Identity Provider (IdP) such as Okta, Azure AD, or Ping Identity, which issues cryptographically signed tokens (SAML assertions, OIDC JWTs) accepted by downstream applications. Multi-Factor Authentication (MFA) requires a second factor—typically a TOTP code from an authenticator app or a hardware FIDO2/WebAuthn key—dramatically reducing the risk of account takeover from phished passwords.

Authorisation determines what an authenticated entity may do. Role-Based Access Control (RBAC) assigns users to roles (viewer, editor, admin) that carry predefined permission sets. Attribute-Based Access Control (ABAC) evaluates dynamic attributes—user department, resource classification, time of day—enabling fine-grained policies. The Principle of Least Privilege demands that every entity receives only the minimum permissions required for its function, minimising blast radius if credentials are compromised.

Privileged Access Management (PAM) tools such as CyberArk, HashiCorp Vault, and BeyondTrust manage highly sensitive credentials: root passwords, service account keys, database connection strings. Just-in-Time (JIT) access provisioning grants elevated privileges only when needed and for a limited time window, automatically revoking them afterward. Secrets management platforms provide centralised, auditable storage for API keys and certificates, eliminating hard-coded credentials in source code.

Knowledge Cycle 3

Artificial Intelligence Fundamentals

AI Fundamentals – Transformers and Large Language Models

The Transformer architecture, introduced by Vaswani et al. in 'Attention Is All You Need' (2017), replaced recurrent networks as the dominant paradigm for sequence modelling. Its core innovation is the self-attention mechanism, which allows each token in a sequence to directly attend to every other token, capturing long-range dependencies without the vanishing-gradient problems that plagued RNNs. Multi-head attention runs multiple attention operations in parallel, each learning different relationship types.

BERT (Bidirectional Encoder Representations from Transformers), released by Google in 2018, demonstrated that pre-training a Transformer on massive unlabelled text corpora (masked language modelling, next-sentence prediction) and fine-tuning on downstream tasks produces state-of-the-art performance across NLP benchmarks. GPT (Generative Pre-trained Transformer) from OpenAI took the complementary approach: causal (left-to-right) language modelling at large scale, producing models that could generate coherent text and perform zero-shot and few-shot tasks.

Scaling laws, empirically established by Kaplan et al. at OpenAI (2020), showed that model performance improves predictably as a power function of model size, dataset size, and compute budget. This insight drove the creation of increasingly large models: GPT-3 (175B parameters, 2020), PaLM (540B, 2022), Llama 2 (70B, open weights, 2023), and Mixtral (sparse mixture-of-experts, 2023). Instruction tuning and Reinforcement Learning from Human Feedback (RLHF) align these models to follow instructions helpfully and safely.

Modern LLM deployment involves several engineering challenges: serving large models efficiently requires techniques such as KV-cache management, continuous batching (vLLM), quantisation (INT8, INT4), and tensor parallelism across multiple GPUs. Context window lengths have grown dramatically—from 2K tokens in GPT-2 to 128K+ in GPT-4 Turbo and Claude 3—enabling retrieval of entire codebases or lengthy documents in a single prompt. Prompt engineering, chain-of-thought reasoning, and tool use (function calling) further extend LLM capabilities.

Vector Databases

Vector Databases – Metadata Filtering and Hybrid Search

Pure vector similarity search returns the closest embeddings in the index regardless of any other attributes. In practice, users almost always want to combine semantic similarity with structured filters:

'find documents similar to this query, but only those written after 2022 and tagged with department=Engineering'. This combination of vector search and metadata filtering is called filtered search or pre-filtered ANN.

Implementing filtered ANN efficiently is non-trivial. A naive approach—compute ANN first, then apply filters—can return too few results if the filter is selective. The opposite approach—filter first, then brute-force search the remaining subset—is accurate but slow when the filtered set is still large. ACORN (Attribute Controlled Retrieval Network), Weaviate's filtered HNSW, and Qdrant's payload indexing use specialised graph traversal or inverted index structures to perform both operations simultaneously with minimal latency penalty.

Hybrid search combines dense vector search with sparse keyword search (BM25 or TF-IDF). Keyword search excels at exact-match queries (product codes, named entities, rare terms that may not be well-represented in the embedding space), while semantic search handles paraphrase and conceptual queries. Reciprocal Rank Fusion (RRF) is a common strategy to merge ranked lists from both retrievers without requiring per-query score calibration. Weaviate's hybrid search, Elasticsearch's ELSER, and Pinecone's sparse-dense index all implement variants of this pattern.

Multi-vector retrieval represents each document with multiple embeddings: one per paragraph, or specialised embeddings for title, abstract, and body. ColBERT (Contextualised Late Interaction over BERT) stores a vector per token and scores query-document pairs using MaxSim across all token pairs, achieving very high recall. The trade-off is storage cost: a 1,000-token document requires 1,000 vectors rather than one. Matryoshka Representation Learning (MRL) trains embedding models so that prefixes of the embedding vector are themselves useful representations, enabling storage-quality trade-offs at query time.

Retrieval-Augmented Generation

RAG – Retrieval Quality and Evaluation

A RAG system is only as good as its retrieval step. If the retriever fails to surface the relevant documents, the LLM cannot generate a correct answer regardless of its capabilities. Retrieval quality is measured using information retrieval metrics: Recall@K (what fraction of relevant documents appear in the top-K results), Precision@K (what fraction of the top-K results are relevant), Mean Reciprocal Rank (MRR), and Normalised Discounted Cumulative Gain (nDCG).

End-to-end RAG evaluation requires annotated question-answer pairs with ground-truth source documents. Frameworks such as RAGAS (Retrieval-Augmented Generation Assessment) provide automated metrics: faithfulness (does the answer contradict the retrieved context?), answer relevance (does the answer address the question?), context precision (are the retrieved chunks relevant?), and context recall (do the retrieved chunks contain sufficient information to answer the question?). ARES and TruLens offer similar capabilities.

Reranking is a common post-retrieval step that re-scores the initial candidate set with a more expensive but more accurate model. A cross-encoder model (e.g., a fine-tuned BERT) receives both the query and each candidate document as a single input and produces a relevance score, unlike bi-encoder models that encode query and document independently. Because cross-encoders are too slow to score the entire corpus, they are applied only to the top-K candidates from the initial retrieval stage—a classic two-stage retrieval pattern.

Query transformation improves recall by reformulating the user query before retrieval. HyDE (Hypothetical Document Embedding) uses an LLM to generate a hypothetical answer to the query, embeds that answer, and retrieves documents similar to it—exploiting the fact that the hypothetical answer is in the same semantic space as the real answer. Step-back prompting asks the LLM to abstract the query to a more general question before retrieving. Multi-query retrieval generates several paraphrases of the original query and takes the union of all retrieved documents.

Cloud Infrastructure

Cloud Infrastructure – Serverless and Event-Driven Architectures

Serverless computing abstracts infrastructure management entirely: developers deploy code (functions) without provisioning or managing servers. The cloud provider handles scaling, patching, high availability, and billing—charged per invocation and per millisecond of execution rather than per hour of server uptime. AWS Lambda, Google Cloud Functions, and Azure Functions are the dominant FaaS (Function-as-a-Service) platforms, each supporting Node.js, Python, Go, Java, and other runtimes.

Serverless functions are inherently stateless and short-lived (maximum execution times of 15 minutes on Lambda). They are well-suited to event-driven workloads: responding to HTTP requests via API Gateway, processing messages from queues (SQS, Pub/Sub), reacting to file uploads (S3 events), or handling database change streams (DynamoDB Streams, Firestore triggers). Cold start latency—the time to initialise a new function instance—is a key concern for latency-sensitive applications, mitigated by provisioned concurrency or keeping functions warm.

Event-driven architecture (EDA) connects services through asynchronous events rather than synchronous API calls, improving loose coupling, resilience, and scalability. An event broker—Apache Kafka, AWS EventBridge, Google Pub/Sub, RabbitMQ—routes events from producers to one or more consumers. The event sourcing pattern stores every state change as an immutable event log, enabling replay, auditing, and temporal queries. The CQRS (Command Query Responsibility Segregation) pattern separates write models (commands that mutate state) from read models (queries), allowing each to scale independently.

Step Functions (AWS), Workflows (Google), and Durable Functions (Azure) provide managed orchestration for multi-step serverless workflows, handling retries, timeouts, branching, and parallel execution. These tools are essential for ML inference pipelines, multi-step data processing, and approval workflows that span multiple services and may run for hours or days. Observability in serverless environments relies on structured logs, distributed traces (AWS X-Ray, Google Cloud Trace), and custom metrics surfaced in dashboards such as Grafana or CloudWatch.

Cybersecurity and Governance

Cybersecurity – Encryption and Data Protection

Encryption is the cornerstone of data protection, transforming plaintext into ciphertext that is unintelligible to anyone without the decryption key. Modern encryption relies on two paradigms: symmetric encryption, where the same key encrypts and decrypts (AES-256 is the standard); and asymmetric encryption, where a public key encrypts and a paired private key decrypts (RSA-4096, Elliptic Curve Cryptography). In practice, systems use asymmetric encryption to exchange a symmetric session key, then use the faster symmetric cipher for bulk data.

Transport Layer Security (TLS) 1.3 is the standard protocol for encrypting data in transit. It provides authentication (the server presents a certificate signed by a trusted Certificate Authority), confidentiality (AES-GCM encryption of the session), and forward secrecy (ephemeral Diffie-Hellman key exchange so that compromising the server's private key does not decrypt past sessions). Certificate Transparency (CT) logs provide public, append-only records of all issued TLS certificates, enabling detection of mis-issued certificates.

Encryption at rest protects data stored on disks, object storage, and databases. Cloud providers offer server-side encryption with provider-managed keys (SSE-S3, Google CMEK default) or customer-managed keys (CMK) stored in hardware security modules (HSMs) via services like AWS KMS, Azure Key Vault, or Google Cloud HSM. Envelope encryption wraps data encryption keys (DEKs) with a key encryption key (KEK), limiting the blast radius of a key compromise. Full-disk encryption (FDE) via BitLocker or LUKS protects physical media.

Tokenisation and data masking are non-cryptographic techniques for protecting sensitive data in non-production environments. Tokenisation replaces sensitive values (credit card numbers, social security numbers) with random tokens that preserve format but carry no exploitable information. Dynamic data masking returns masked values to unauthorised queries while preserving full values for authorised users. These techniques are essential for compliance with PCI-DSS (payment card data) and HIPAA (health information) regulations.

Knowledge Cycle 4

Artificial Intelligence Fundamentals

AI Fundamentals – AI in Industry Verticals

Healthcare AI encompasses diagnostic imaging analysis (detecting tumours, fractures, retinal abnormalities from X-rays, CT scans, and MRIs), clinical decision support (recommending treatment options based on patient history and clinical guidelines), drug discovery (predicting molecular binding affinity with target proteins), and administrative automation (medical coding, prior authorisation, appointment scheduling). FDA-cleared AI devices include ECG analysis algorithms and diabetic retinopathy screening tools deployed at scale.

Financial services AI powers fraud detection (transaction anomaly scoring in real time), credit risk modelling (using alternative data such as spending patterns and mobile metadata alongside traditional credit bureau data), algorithmic trading (high-frequency strategies, portfolio optimisation), regulatory compliance (anti-money laundering transaction monitoring, sanctions screening), and personalised financial advisory (robo-advisors, personalised product recommendations). Explainability requirements under regulations like the EU AI Act and US Fair Lending laws constrain model choices.

Manufacturing and supply chain AI includes predictive maintenance (forecasting equipment failure from sensor data to schedule proactive maintenance before breakdown), quality inspection (computer vision systems detecting surface defects faster and more consistently than human inspectors), demand forecasting (incorporating external signals such as weather and economic indicators), and autonomous mobile robots (AMRs) for warehouse picking and pallet movement. Digital twins—virtual replicas of physical assets—enable simulation-based optimisation of factory layouts and logistics networks.

In software engineering, AI-powered tools have reached widespread adoption. GitHub Copilot, Amazon CodeWhisperer, and Cursor AI generate, complete, and refactor code within IDEs. LLM-based agents can autonomously run test suites, interpret failure messages, propose patches, and open pull requests. Automated code review tools scan for security vulnerabilities (SAST), code smells, and licence compliance issues. Natural language to SQL (text-to-SQL) tools allow business analysts to query databases without writing SQL.

Vector Databases

Vector Databases – Operational Considerations at Scale

Running a vector database in production requires thinking beyond indexing algorithms to address operational concerns: availability, durability, consistency, and cost. For high availability, vector

databases replicate data across multiple nodes. Weaviate and Qdrant support replication factors, meaning each shard is stored on N nodes; queries can be served by any replica, and the system tolerates N-1 node failures. During write operations, consistency levels (ONE, QUORUM, ALL) trade latency for durability guarantees.

Index build time and re-indexing strategy are critical at large scale. HNSW graphs must be built incrementally as new vectors are inserted; most implementations allow online insertions with amortised re-balancing. However, rebuilding an index from scratch—sometimes necessary after bulk imports or algorithm changes—can take hours for billion-scale collections. Segment-based architectures (similar to Lucene in Elasticsearch) write immutable segments that are periodically merged, enabling parallelised builds and fast compaction.

Cost management in cloud-hosted vector databases involves balancing storage tier, index type, and replica count. Pinecone's pod-based pricing charges per pod-hour; pod types (s1, p1, p2) trade storage density for query throughput. For cost-sensitive workloads, quantised indices (INT8, binary) reduce storage by 4-32x at the cost of 1-5% recall degradation. Offloading rarely-queried collections to cold storage with lazy loading (DiskANN) can reduce costs significantly.

Monitoring a vector database in production requires tracking: query latency (p50, p95, p99), recall (estimated or measured against a ground-truth evaluation set), index freshness (lag between document creation and availability in the index), memory utilisation, and shard balance. Embedding drift—the phenomenon where the distribution of incoming embeddings shifts over time relative to the distribution used to build the index—can silently degrade recall without obvious error signals, making periodic recall audits important.

Retrieval-Augmented Generation

RAG – Advanced Patterns: Agentic and Graph RAG

Standard RAG performs a single retrieve-then-generate cycle. Agentic RAG extends this with iterative retrieval: the LLM decides whether its retrieved context is sufficient, formulates follow-up queries if not, and continues retrieving until it has enough information to answer confidently. This pattern, implemented in frameworks like LangGraph, LlamaIndex's ReAct agent, and AutoGPT, is particularly valuable for complex, multi-hop questions that cannot be answered from a single retrieved passage.

Multi-hop retrieval addresses questions that require chaining together evidence from multiple documents. For example, answering 'What was the revenue of the company that acquired Startup X in 2022?' requires first retrieving information about the acquisition, then retrieving financial data about the acquirer. IRCoT (Interleaving Retrieval with Chain-of-Thought) prompts the LLM to alternate between reasoning steps and retrieval actions, building up a reasoning chain grounded in retrieved evidence at each step.

Graph RAG organises the knowledge base as a knowledge graph—entities (people, companies, concepts) as nodes and relationships as edges—and traverses the graph to gather multi-hop evidence. Microsoft's GraphRAG system builds a community-detection hierarchy over entities extracted from documents, enabling both local queries (about specific entities) and global queries (about themes across the entire corpus). Graph RAG excels when relationships between entities are as important as the content of individual documents.

Adaptive RAG uses a lightweight classifier to route queries to different retrieval strategies based on their complexity. Simple factoid questions may not need retrieval at all (the LLM's parametric memory suffices). Single-hop questions benefit from standard RAG. Complex, multi-document questions are routed to agentic or graph RAG pipelines. This routing reduces latency and cost for simple queries while ensuring accuracy for complex ones. FLARE (Forward-Looking Active Retrieval) triggers retrieval on-the-fly whenever the LLM generates a low-confidence token.

Cloud Infrastructure

Cloud Infrastructure – Data Engineering and Lake Architectures

Modern data architectures separate storage from compute, enabling independent scaling of each. The Data Lake pattern stores raw, unprocessed data in object storage (S3, GCS, Azure Data Lake) in open formats (Parquet, ORC, Avro), keeping costs low and preserving flexibility for future processing. ETL/ELT pipelines transform and load data into analytical warehouses (Snowflake, BigQuery, Redshift) for structured querying, or into feature stores (Feast, Tecton) for ML workloads.

Apache Spark is the dominant distributed data processing engine, enabling parallel computation over petabyte-scale datasets using a functional API (map, filter, join, aggregate) that translates to an optimised physical execution plan. Spark Structured Streaming extends this to continuous stream processing with exactly-once semantics. Apache Flink offers lower-latency stream processing with native event-time semantics and stateful operators, preferred for real-time fraud detection and monitoring pipelines.

The Data Lakehouse pattern combines the low cost and openness of data lakes with the ACID transactions and schema enforcement of data warehouses. Table formats—Delta Lake (Databricks), Apache Iceberg (Netflix, Apple), Apache Hudi (Uber)—add transaction logs on top of object storage Parquet files, enabling upserts, deletes, time-travel queries, and schema evolution without sacrificing compatibility with Spark, Trino, and other engines. The Lakehouse has become the dominant architecture for large-scale analytics.

Data mesh is an organisational and architectural pattern that decentralises data ownership to domain teams. Each domain (e.g., Orders, Customers, Inventory) owns, produces, and maintains its data as a product with a published schema, SLO, and self-serve access controls. A central data platform team provides shared infrastructure—storage, cataloguing (Apache Atlas, Amundsen, DataHub), access

control, and lineage tracking—but does not own the data itself. The federated governance model balances autonomy with cross-domain discoverability.

Cybersecurity and Governance

Cybersecurity – Incident Response and Recovery

Despite preventive controls, security incidents will occur. An Incident Response (IR) plan defines the structured process for detecting, containing, eradicating, and recovering from security events. The NIST SP 800-61 framework organises IR into four phases: Preparation (training, tooling, playbooks), Detection and Analysis (identifying and triaging events), Containment, Eradication, and Recovery (stopping the spread, removing malware, restoring systems), and Post-Incident Activity (root cause analysis, lessons learned).

Preparation is the most impactful phase. IR teams should pre-build playbooks for common incident types—ransomware, credential compromise, data exfiltration, DDoS—and conduct tabletop exercises and red/blue team drills at least annually. A dedicated IR retainer with a third-party firm (CrowdStrike, Mandiant, Palo Alto Unit 42) provides surge capacity and forensic expertise for major incidents. Communication templates for customer, regulator, and board notifications should be prepared in advance.

Containment strategies depend on the nature of the incident. Network segmentation (blocking lateral movement via firewall rules, VLAN changes) prevents an attacker from spreading from an initially compromised host. Disabling compromised accounts and rotating credentials stops authentication-based attacks. For ransomware, the critical decision is whether to isolate affected systems immediately (halting operations) or to monitor for a period to understand the full scope of compromise before acting—a trade-off between business continuity and forensic completeness.

Forensic evidence preservation is legally and operationally critical. Digital forensics professionals collect disk images, memory dumps, network captures, and log archives from affected systems before remediation destroys evidence. Chain-of-custody documentation ensures evidence is admissible if criminal prosecution follows. Log retention policies must ensure that relevant logs (authentication, network flow, endpoint telemetry) are retained for at least 12 months; many regulations require 24+ months. A SIEM with centralised log ingestion and tamper-evident storage is essential.

Knowledge Cycle 5

Artificial Intelligence Fundamentals

AI Fundamentals – AI Ethics, Bias, and Fairness

AI systems trained on historical data inherit the biases embedded in that data, which can perpetuate and amplify social inequities at scale. Bias in ML refers to systematic errors that produce unfair outcomes for particular demographic groups. Types of bias include historical bias (past discrimination reflected in training data), representation bias (underrepresentation of minority groups), measurement bias (lower-quality data for certain groups), aggregation bias (a single model applied to populations with different underlying distributions), and evaluation bias (test sets that do not reflect real-world diversity).

Fairness in ML is formally defined in multiple, often mutually exclusive ways. Demographic parity requires that positive prediction rates are equal across groups. Equalised odds requires equal true positive and false positive rates across groups. Individual fairness requires that similar individuals receive similar predictions. The impossibility theorem (Chouldechova, 2017; Kleinberg et al., 2016) proves that many fairness criteria cannot be simultaneously satisfied when base rates differ across groups, forcing practitioners to make explicit trade-off decisions.

Techniques for bias mitigation fall into three categories. Pre-processing methods modify the training data: re-sampling under-represented groups, re-weighting examples, or removing sensitive attributes. In-processing methods add fairness constraints to the optimisation objective during training (adversarial debiasing, fairness regularisation). Post-processing methods adjust model outputs—shifting decision thresholds per group or calibrating score distributions—after training. IBM's AI Fairness 360 and Google's Fairness Indicators provide open-source toolkits for measuring and mitigating bias.

Explainability and transparency are increasingly required by regulation. The EU AI Act mandates conformity assessments and transparency obligations for high-risk AI systems (credit scoring, employment screening, critical infrastructure). LIME (Local Interpretable Model-Agnostic Explanations) and SHAP (SHapley Additive exPlanations) are widely used post-hoc explanation methods that attribute each prediction to input features. Model cards (Mitchell et al., 2019) and datasheets for datasets (Gebru et al., 2018) are documentation frameworks for communicating model performance, limitations, and intended use.

Vector Databases

Vector Databases – Embedding Models and Dimensionality

The quality of a vector database's retrieval is determined as much by the embedding model as by the indexing algorithm. An embedding model encodes a query and a document into the same vector space; their similarity is then used as a proxy for relevance. Choosing the right model requires considering: the domain (general web text vs. scientific papers vs. code), the language (multilingual vs. English-only), the input length (sentence-level vs. document-level), and the performance on standardised benchmarks such as MTEB (Massive Text Embedding Benchmark).

OpenAI's text-embedding-3-large (3072 dimensions, reduced via MRL to 256-3072), Cohere Embed v3 (1024 dimensions, with input-type specialisation for queries vs. documents), and open-source models such as E5-large-v2, GTE-Qwen2-7B, and nomic-embed-text-v1.5 consistently top MTEB leaderboards. For code retrieval, CodeBERT and UniXcoder produce embeddings that cluster semantically similar code across languages. For multimodal retrieval, CLIP and its successors (SigLIP, OpenCLIP) embed images and text in the same space.

Higher embedding dimensionality generally improves retrieval quality at the cost of storage and compute. A 1536-dimension embedding requires 6KB per vector (float32); storing 10 million embeddings requires ~60GB of RAM with no overhead. Dimensionality reduction techniques—PCA, UMAP, or MRL truncation—can reduce footprint significantly with modest recall loss. Binary embeddings (1 bit per dimension) reduce storage by 32x and enable ultra-fast Hamming distance computation, with recent research showing surprisingly competitive recall.

Fine-tuning embedding models on domain-specific data substantially improves retrieval quality in specialised domains. Contrastive learning with hard negatives—training the model to score relevant (query, document) pairs higher than challenging negatives—is the most effective fine-tuning strategy. Datasets of (query, relevant document, hard negative) triplets can be constructed from user click logs, expert annotations, or LLM-generated synthetic data. Sentence Transformers' training utilities and Hugging Face's SetFit framework provide accessible fine-tuning pipelines.

Retrieval-Augmented Generation

RAG – Production Engineering and Observability

Deploying a RAG system to production requires the same engineering rigour as any distributed system: SLOs, monitoring, CI/CD for the knowledge base, and graceful degradation under load. Define latency SLOs for each component: embedding time (typically 10-50ms per query), vector search (5-50ms for millions of vectors), LLM generation (highly variable, 500ms–10s depending on model and output length). Set error budget policies and alerting thresholds accordingly.

Knowledge base freshness is a critical operational concern. Source documents are created, updated, and deleted continuously; the vector index must reflect the latest state. An event-driven pipeline—triggered by document creation/update webhooks or CDC (change data capture) from a document store—re-chunks, re-embeds, and upserts changed documents within minutes of the source

change. Deleted documents must be explicitly removed from the index; stale retrieval of deleted documents is a common and subtle bug.

Prompt engineering for RAG involves carefully structuring the system prompt, the retrieved context, and the user query. Best practices include: placing retrieved context before the question (not after), explicitly instructing the model to acknowledge when context does not support an answer, using XML or Markdown delimiters to clearly separate retrieved chunks, and including metadata (source title, date) alongside chunk content so the model can cite sources. Context compression—summarising retrieved chunks before passing them to the LLM—reduces cost and latency when documents are verbose.

Observability for RAG requires logging inputs and outputs at every stage: user query, transformed queries, retrieved chunk IDs and scores, LLM prompt, and LLM response. Structured logging enables downstream analysis: computing retrieval hit rate, measuring correlation between retrieval scores and answer quality, detecting queries that frequently produce low-confidence answers, and identifying data gaps in the knowledge base. Tools such as LangSmith, Arize Phoenix, and WhyLabs provide purpose-built RAG observability dashboards.

Cloud Infrastructure

Cloud Infrastructure – Cost Optimisation Strategies

Cloud cost optimisation begins with visibility: understanding where money is being spent before attempting to reduce it. Cloud providers offer native cost management tools (AWS Cost Explorer, GCP Billing, Azure Cost Management) that break down spending by service, region, tag, and account. FinOps—the practice of bringing financial accountability to cloud spending—recommends tagging all resources with cost-centre, team, and environment labels, enabling chargeback to internal teams and surfacing inefficiencies.

Compute cost optimisation leverages pricing models and right-sizing. On-demand instances are the most expensive; Reserved Instances (1- or 3-year commitments, up to 72% discount on AWS) and Savings Plans (flexible commitment to a \$/hour spend) dramatically reduce costs for steady-state workloads. Spot/preemptible instances (up to 90% discount for spare capacity) are ideal for fault-tolerant, interruptible batch workloads—Spark jobs, ML training, CI pipelines. Right-sizing analysis compares CPU and memory utilisation against provisioned capacity and downsizes over-provisioned instances.

Storage cost optimisation leverages tiering. Object storage (S3, GCS) offers multiple tiers: Standard (low latency, high cost), Intelligent-Tiering (automatic movement between tiers based on access patterns), Glacier Instant Retrieval, Glacier Flexible Retrieval, and Glacier Deep Archive (lowest cost, hours-to-days retrieval). Lifecycle policies automate transitions between tiers—moving data to Glacier after 90 days, deleting it after 7 years. Enabling S3 Intelligent-Tiering for datasets with unpredictable access patterns typically reduces storage costs by 20-40%.

Network egress—traffic leaving the cloud provider's network—is often the most surprising cost. Data transfer between AWS regions costs \$0.02/GB; data transfer to the internet costs \$0.09/GB on AWS. Strategies to minimise egress include deploying consumers in the same region as producers, using VPC endpoints (private connectivity to AWS services that avoids internet egress), compressing data before transfer, and using CDNs (CloudFront, Cloudflare) to cache static content close to end users. A distributed tracing system that tags spans with data transfer volume helps attribute network costs.

Cybersecurity and Governance

Cybersecurity – Secure Development Lifecycle (SDL)

The Secure Development Lifecycle (SDL) integrates security practices into every phase of software development—from requirements gathering through design, implementation, testing, deployment, and maintenance—rather than treating security as a bolt-on audit at release time. Microsoft's SDL, published in 2004 after the Security Push that produced Windows XP SP2, is the canonical reference and has been adopted (with adaptations) across the industry.

Threat modelling during the design phase systematically identifies potential security issues before code is written. The STRIDE framework (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) categorises threats; DREAD scores their risk. Data flow diagrams (DFDs) map how data moves through a system, highlighting trust boundaries where security controls must be applied. Tools such as Microsoft Threat Modeling Tool and OWASP Threat Dragon automate parts of this process.

Secure coding practices prevent the introduction of common vulnerabilities. OWASP Top 10 lists the most critical web application security risks: injection (SQL, OS command, LDAP), broken authentication, sensitive data exposure, XML External Entities (XXE), broken access control, security misconfiguration, cross-site scripting (XSS), insecure deserialization, using components with known vulnerabilities, and insufficient logging/monitoring. Code reviews, pair programming, and linter rules enforce secure patterns. Input validation (whitelisting allowed characters) and parameterised queries prevent injection attacks.

DevSecOps integrates security tooling into the CI/CD pipeline. Static Application Security Testing (SAST) tools—Semgrep, SonarQube, Checkmarx—analyse source code for vulnerabilities without executing it. Dynamic Application Security Testing (DAST) tools—OWASP ZAP, Burp Suite—execute the application and probe it for vulnerabilities at runtime. Software Composition Analysis (SCA) tools—Snyk, Dependabot, FOSSA—scan open-source dependencies for known CVEs and licence violations. Container image scanning (Trivy, Gype) analyses Docker images before deployment.

Appendix: Recommended Chunking Strategies

Chunking is one of the most important aspects of RAG system quality. The table below summarises the primary strategies, their recommended parameters, and their ideal use cases. Recursive chunking with overlap is typically the best starting point for general corpora; semantic chunking is preferred when topic coherence is critical; sentence-window retrieval is optimal for question-answering over densely factual documents.

Technique	Chunk Size	Overlap	Use Case	Pros	Cons
Recursive Character	500–1000 chars	100–200 chars	General RAG	Fast, simple, widely supported	Ignores semantic boundaries
Semantic (embedding)	Variable	Minimal	Research corpora	Respects topic coherence	Slower build time, thresholding
Sentence Window	3–5 sentences	1 sentence	QA systems	High retrieval precision	Requires window expansion
Parent Document	Small index, large retrieval	None	Long-form docs	Rich context for LLM	Higher storage overhead
Fixed Size	256–512 tokens	10–20%	High-volume ingest	Predictable, easy to parallelise	May split mid-sentence

Example LangChain Chunking Code

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,
    chunk_overlap=150,
    separators=["\n\n", "\n", ".", " ", ""]
)

chunks = splitter.split_documents(documents)
```